

# Indeed-LMS

## Complete Codebase Analysis & Dependency Resolution Report

Senior Python/Django Developer & DevOps Specialist

August 22, 2026

### Contents

|                                                              |                   |
|--------------------------------------------------------------|-------------------|
| <a href="#">1 Executive Summary</a>                          | <a href="#">3</a> |
| <a href="#">2 Repository Structure</a>                       | <a href="#">3</a> |
| <a href="#">3 Django Architecture</a>                        | <a href="#">3</a> |
| <a href="#">4 Technology Stack</a>                           | <a href="#">4</a> |
| <a href="#">5 Dependency Analysis</a>                        | <a href="#">4</a> |
| <a href="#">6 requirements.txt vs requirements_rohit.txt</a> | <a href="#">4</a> |
| <a href="#">7 Python Version Analysis</a>                    | <a href="#">5</a> |
| <a href="#">8 Dependency Conflicts</a>                       | <a href="#">5</a> |
| <a href="#">9 Environment Variables</a>                      | <a href="#">5</a> |
| <a href="#">10 Authentication Architecture</a>               | <a href="#">5</a> |
| <a href="#">11 Database Architecture</a>                     | <a href="#">5</a> |
| <a href="#">12 AI/ML Architecture</a>                        | <a href="#">6</a> |
| <a href="#">13 External Services</a>                         | <a href="#">6</a> |
| <a href="#">14 API Architecture</a>                          | <a href="#">6</a> |
| <a href="#">15 Current Startup Errors</a>                    | <a href="#">6</a> |
| <a href="#">16 Missing Configuration</a>                     | <a href="#">7</a> |
| <a href="#">17 Correct Local Development Setup</a>           | <a href="#">7</a> |
| <a href="#">18 Recommended Dependency Strategy</a>           | <a href="#">7</a> |
| <a href="#">19 Security Issues</a>                           | <a href="#">7</a> |
| <a href="#">20 Technical Debt</a>                            | <a href="#">7</a> |



# 1 Executive Summary

**Indeed-LMS** is an advanced Learning Management System (LMS) backend. In addition to standard LMS features (e.g. course catalogs, video content progress tracking, student enrollment, assessments, blog posts, and certificates), the core value proposition of this backend is its **AI-powered interactive mock interview preparation system**.

The mock interview engine consists of:

1. **Interactive WebSocket Chat:** Powered by Django Channels (`app/consumers.py`), allowing real-time voice chunk transmission, converting it to text via Google Speech-to-Text (`speech_recognition`), and evaluating answers using the OpenAI API.
2. **Real-time WebRTC Voice Agent:** Built using the `livekit-agents` framework (`app/interview_bot/`), utilizing OpenAI's `gpt-4o-mini` for completions and Deepgram/OpenAI/Silero plugins for high-fidelity audio streams.
3. **Retrieval-Augmented Generation (RAG):** Processes uploaded resumes and job descriptions (JDs), chunks and embeds them via OpenAI embeddings (`text-embedding-3-small`), and stores the resulting index as a serialized Annoy vector database (`.pkl`) in the MySQL database.

# 2 Repository Structure

Below is an overview of the directory hierarchy and key modules in the repository:

- **LMS/:** The root Django project configuration folder. Contains `settings.py`, `urls.py`, `asgi.py`, `wsgi.py`, and `celery.py`.
- **app/:** The primary application code. Houses database models, API views, serializers, websocket consumers, and business logic.
- **app/interview\_bot/:** LiveKit-specific worker scripts (`main.py`) and vector RAG build scripts (`built_rag.py`).
- **app/interview\_main/:** Evaluator prompts and orchestration classes for mock interviews.
- **app/rag/:** Code managing Annoy indexing, vector storage, and chunking configurations.
- **dashboard/:** Django application handling admin APIs and administrative dashboard actions.
- **templates/:** Django HTML templates, including the client page for WebSocket recording (`interview_page.html`).
- **static/ & media/:** Directories serving assets and storing uploaded documents.

# 3 Django Architecture

- **Django Version:** 5.2.13
- **ASGI/WSGI Entrypoints:** ASGI is handled by Daphne for Channels protocol routing; WSGI handles standard page requests.
- **Middlewares:** Pinned to standard Django middlewares with `CorsMiddleware` enabled to allow requests from external clients.
- **App Loading Pattern:** On launch, Django loads settings, imports routing patterns, resolves application models, and initializes the URL configuration module.

## 4 Technology Stack

The backend relies on the following core components:

- **Python Interpreter:** Version 3.11.9
- **Core Web Stack:** Django 5.2.13, DRF 3.17.1, Daphne 3.0.2, Channels 4.3.2
- **Database Engines:** MySQL (Production) & SQLite (Local Dev fallback)
- **Asynchronous Worker Stack:** Celery 5.6.3 & Redis (broker / channel layer)
- **AI Integrations:** OpenAI API (embeddings & chat), LiveKit Agents 1.5.8, Deepgram, Silero, Annoy
- **E-commerce / Services:** Razorpay SDK, Msg91 API, SMTP Gmail backend

## 5 Dependency Analysis

The table below compares the original requirements with the working local virtualenv snapshot (column headers: Original = `requirements.txt`, Rohit = `requirements_rohit.txt`):

| Package               | Original   | Rohit      | Installed     | Conflict? | Used? |
|-----------------------|------------|------------|---------------|-----------|-------|
| protobuf              | 6.33.6     | 5.29.5     | 5.29.5        | Yes       | No    |
| mediapipe             | 0.10.14    | Not Pinned | Not Installed | Yes       | No    |
| numpy                 | 1.26.4     | 1.26.4     | 1.26.4        | No        | Yes   |
| tensorflow            | 2.16.1     | Not Pinned | Not Installed | Yes       | No    |
| tensorflow-intel      | 2.16.1     | Not Pinned | Not Installed | Yes       | No    |
| keras                 | 3.1.1      | Not Pinned | Not Installed | Yes       | No    |
| torch                 | Not Pinned | 2.11.0     | 2.11.0        | No        | No    |
| transformers          | Not Pinned | 5.5.4      | 5.5.4         | No        | No    |
| langchain             | 0.1.0      | 0.1.0      | 0.1.0         | No        | No    |
| langchain-community   | 0.0.16     | 0.0.16     | 0.0.16        | No        | No    |
| langchain-core        | 0.1.23     | 0.1.23     | 0.1.23        | No        | No    |
| aiohttp               | 3.13.5     | 3.13.5     | 3.13.5        | No        | Yes   |
| async-timeout         | 5.0.1      | 4.0.3      | 4.0.3         | Yes       | No    |
| django                | 5.2.13     | 5.2.13     | 5.2.13        | No        | Yes   |
| djangorestframework   | 3.17.1     | 3.17.1     | 3.17.1        | No        | Yes   |
| channels              | 4.3.2      | 4.3.2      | 4.3.2         | No        | Yes   |
| celery                | 5.6.3      | 5.6.3      | 5.6.3         | No        | Yes   |
| redis                 | 7.3.0      | 7.3.0      | 7.3.0         | No        | Yes   |
| pydantic              | 2.13.1     | 2.13.1     | 2.13.1        | No        | No    |
| sentence-transformers | 5.4.1      | 5.4.1      | 5.4.1         | No        | No    |

## 6 requirements.txt vs requirements\_rohit.txt

1. **Genesis of requirements\_rohit.txt:** This file represents a dependency snapshot captured using `pip freeze`. It contains transitive dependencies that ensure reproducible setup.
2. **Unused heavy libraries:** Large machine learning packages (TensorFlow, Keras, Mediapipe, ONNX Runtime, Opentelemetry) are unreferenced in the source code. They are dead dependencies and were safely excluded in `requirements_rohit.txt`, solving pip resolution conflicts.
3. **Action:** `requirements.txt` should be replaced with the cleaned set.

## 7 Python Version Analysis

The virtual environment currently utilizes **Python 3.11.9**. All core libraries, including Django 5.x, DRF, Daphne, Channels, Celery, and LiveKit plugins run without warnings on Python 3.11.9. There is no software requirement that necessitates downgrading to Python 3.10.

## 8 Dependency Conflicts

- **The Conflict:** `mediapipe 0.10.14` requires `protobuf>=4.25.3,<5`. Pinned `protobuf==6.33.6` violates this condition.
- **Impact:** Pip installation aborts since it cannot resolve constraints.
- **Audit Findings:** Since `Mediapipe` and `TensorFlow` are never imported, the cleanest solution is to omit them, allowing `protobuf` to resolve to version `5.29.5` as required by `LiveKit`.

## 9 Environment Variables

The following table details the variables parsed from settings and scripts:

| Variable                 | Used In         | Required? | Purpose                   | Local Dev Value         |
|--------------------------|-----------------|-----------|---------------------------|-------------------------|
| DJANGO_SECRET_KEY        | LMS/settings.py | Yes       | Django cryptographic key  | Local dev secret        |
| ENVIRONMENT              | LMS/settings.py | Yes       | Enables debug             | local                   |
| AUTHENTICATION_CLASS     | LMS/settings.py | Yes       | Configures DRF auth class | CookieJWTAuthentication |
| DB_NAME / USER / PASS    | LMS/settings.py | Yes       | MySQL configurations      | Local DB configurations |
| OPENAI_API_KEY           | Python modules  | Yes       | Generates embeddings      | sk-proj-...             |
| LIVEKIT_URL              | LMS/settings.py | No        | WebRTC server endpoint    | http://localhost:7880   |
| LIVEKIT_API_KEY / SECRET | LMS/settings.py | No        | WebRTC authentication     | Local dev key/secret    |
| CHAT_SOCKET_URL          | LMS/settings.py | No        | WebSockets URL            | ws://127.0.0.1:8000/... |

## 10 Authentication Architecture

Authentication follows a dual-mode JWT pattern:

- Login verification returns `SimpleJWT` tokens.
- Tokens are written to client headers and saved in an `HTTPOnly` cookie (`access_token`).
- `CookieJWTAuthentication` inspects the auth header first; if blank, it extracts the cookie.
- Permissions are checked globally using `IsAuthenticated`.

## 11 Database Architecture

- **Production Engine:** MySQL is set by default.

- **Local SQLite DB:** A local file `db.sqlite3` exists in the root. Its migration history contains old records (0002 to 0027) which were deleted from disk.
- **Local dev strategy:** To avoid installing MySQL locally, settings can conditionally fall-back to SQLite. The old local `db.sqlite3` file should be backed up, and clean migrations should be applied.

## 12 AI/ML Architecture

AI components are fully hosted API engines (OpenAI & LiveKit Cloud). Vector indexing is built locally via PyMuPDF (text extraction) and Annoy (vector indexing). Index files are written to database blobs as `.pkl` files, making GPU requirements unnecessary for local environments.

## 13 External Services

- **OpenAI API:** Mandatory for LLM evaluation and generating embeddings.
- **Redis & Celery:** Required for message broker queues and Channel layers.
- **LiveKit server:** Required for real-time voice streaming tests.

## 14 API Architecture

Core REST endpoints are defined under `/api/`:

- `/api/accounts/register` & `/api/accounts/verify-otp` manage authentication.
- `/api/start-interview/` initiates RAG pipeline and returns a LiveKit session token.
- `/api/check-chat-access/` verifies session credits.
- WebSockets connect to `ws/interview/` for interactive speech-to-text chat.

## 15 Current Startup Errors

1. **AttributeError: 'NoneType' object has no attribute 'rsplit'** in `rest_framework.settings.py`:
  - *Root Cause:* `AUTHENTICATION_CLASS` environment variable is missing, rendering the DRF classes tuple as `(None,)`.
  - *Fix:* Add `AUTHENTICATION_CLASS=app.authentication.CookieJWTAuthentication` to `.env`.
2. **openai.OpenAIError: The api\_key client option must be set:**
  - *Root Cause:* Global OpenAI instantiation at import time in `built_rag.py` fails if `OPENAI_API_KEY` is not in the environment.
  - *Fix:* Set a valid `OPENAI_API_KEY` in `.env`.

## 16 Missing Configuration

A local `.env` file must be created:

```
1 DJANGO_SECRET_KEY=local_development_secret_string
2 ENVIRONMENT=local
3 AUTHENTICATION_CLASS=app.authentication.CookieJWTAuthentication
4 OPENAI_API_KEY=sk-proj-YOUR_KEY_HERE
5 CHAT_SOCKET_URL=ws://127.0.0.1:8000/ws/interview/
```

## 17 Correct Local Development Setup

1. Configure local `.env` file.
2. Configure `LMS/settings.py` to fallback to SQLite when `ENVIRONMENT == 'local'`.
3. Ensure Redis is running (port 6379).
4. Back up the existing `db.sqlite3` and run `python manage.py migrate`.
5. Run `python manage.py check`.
6. Start server: `python manage.py runserver`.

## 18 Recommended Dependency Strategy

- Use `requirements_rohit.txt` without TensorFlow/Mediapipe.
- Retain Python 3.11.9.
- Retain Protobuf 5.29.5.
- Remove MediaPipe as it is never imported in the codebase.

## 19 Security Issues

- **Hardcoded auth keys:** In `app/send_otp.py`, the Msg91 API authkey is hardcoded.
- **SMTP credentials:** Gmail credentials are hardcoded in `LMS/settings.py`.
- **CORS Settings:** Allow All origins is enabled.

## 20 Technical Debt

- Global instantiations of the OpenAI client crash start commands when environment keys are missing.
- Deleted migration files on disk conflict with database history.

## 21 Exact Next Steps

1. Define local `.env` variables.
2. Add conditional SQLite database fallback for local environments to `LMS/settings.py` (pending approval).
3. Refactor module-level OpenAI client setup to use lazy function instantiation (pending approval).